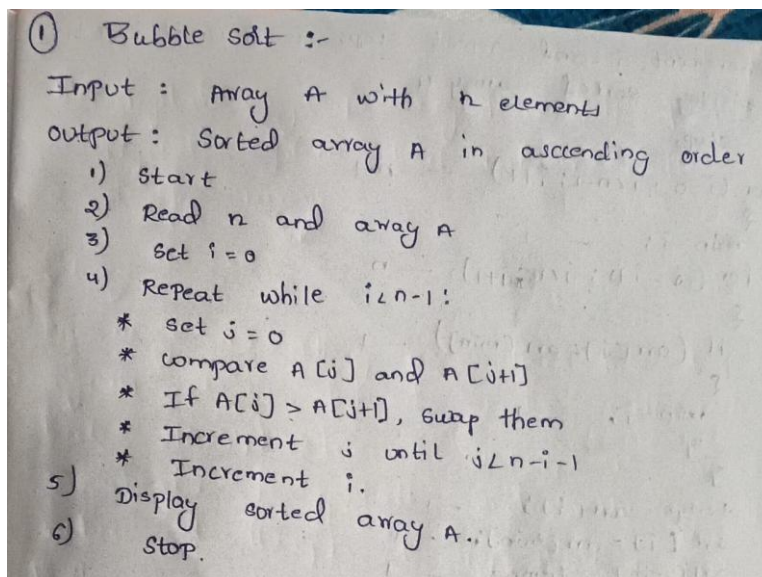


PRACTICAL - 1

AIM : Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

BUBBLE SORT ALGORITHM



PROGRAM:

```
#include <iostream>
using namespace std;
class bubblesort {
public:
    int size, arr[100], i;
    void arrayinput()

.... {      cout << "Enter array size: ";
cin >> size;      cout << "Enter array
elements:\n";      for (i = 0; i < size;
i++)
    {      cin
>> arr[i];
    }
```



```
    }    void
printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }    cout
<< endl;
}
void bubblelogic()
{    int j, temp;    for
(i = 0; i < size - 1; i++)
{    for (j = 0; j < size - i -
1; j++)
{
    if (arr[j] > arr[j + 1])
    {
temp = arr[j];
arr[j] = arr[j + 1];    arr[j
+ 1] = temp;
    }
    }
}
} }; int
main() {
    bubblesort bs;
bs.arrayinput();    cout <<
"Before sorting: ";
bs.printarray();
bs.bubblelogic();    cout <<
"After sorting: ";
bs.printarray();    return 0;
}
```

OUTPUT :

```
Enter array size: 4
Enter array elements:
5
4
6
9
Before sorting: 5 4 6 9
After sorting: 4 5 6 9

...Program finished with exit code 0
Press ENTER to exit console.
```

TIME COMPLEXITY : Step Count / Frequency Count Method)

best case :

best case :
Bubble sort
first Pass $\rightarrow n-1$
no swapping
loops stops
 $T(n) = n-1$
Best case = $O(n)$

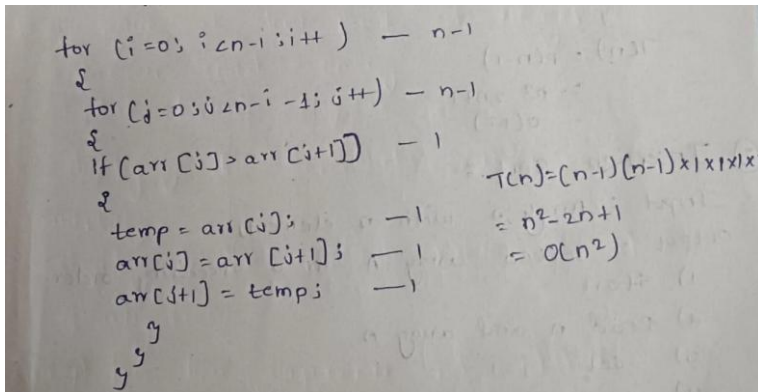
average case :

```
for (i=0; i<n-1; i++) — n-1
{
  for (j=0; j<n-i-1; j++) — n-1
  {
    if (arr[j] > arr[j+1]) — 1
    {
      temp = arr[j]; — 1
      arr[j] = arr[j+1]; — 1
      arr[j+1] = temp; — 1
    }
  }
}
```

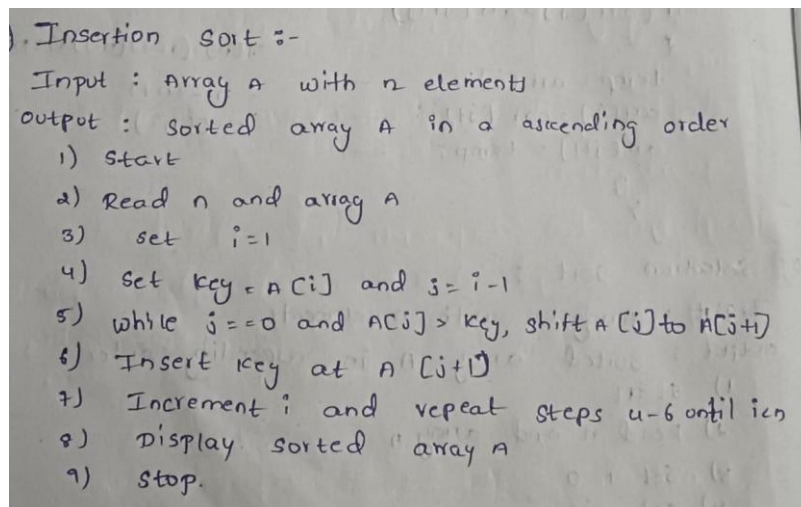
$T(n) = (n-1)(n-1) \times 1 \times 1 \times 1$
 $= n^2 - 2n + 1$
 $= O(n^2)$



Worst case :



INSERTION ALGORITHM



PROGRAM:

```
#include <iostream> using
namespace std;

class InsertionSort { public:
int size, arr[100], i, j, temp;

void arrayinput()
{ cout << "Enter array
size: "; cin >> size;
```



```
        cout << "Enter array elements: ";
for (i = 0; i < size; i++)

    {
        cin
    >> arr[i];
    }

void printarray()
{

    for (i = 0; i < size; i++)
    {
        cout <<
arr[i] << " ";
    }
    cout
<< endl;
}

void insertionsortlogic()
{
    for (i = 1; i < size;
i++)
    {
        temp
= arr[i];
j = i - 1;

        while (j >= 0 && arr[j] > temp)
        {
            arr[j +
1] = arr[j];
            j--;
        }

        arr[j + 1] = temp;
    }
}

int main()
{
    InsertionSort is;

    is.arrayinput();
```

```
cout << "Before sorting: ";  
is.printarray();
```

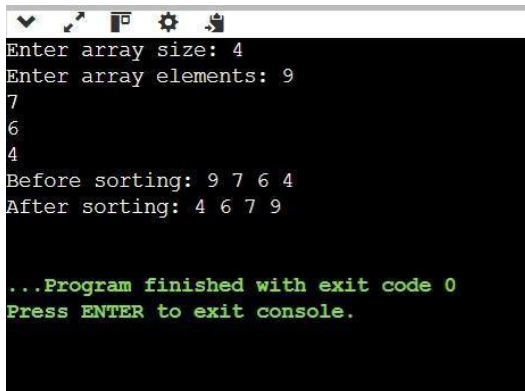
```
is.insertionsortlogic();
```

```
cout << "After sorting: ";  
is.printarray();
```

```
return 0;
```

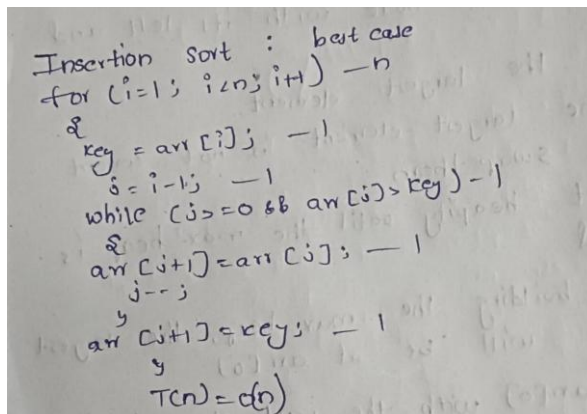
```
}
```

OUTPUT :



```
Enter array size: 4  
Enter array elements: 9  
7  
6  
4  
Before sorting: 9 7 6 4  
After sorting: 4 6 7 9  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

TIME COMPLEXITY : Step Count / Frequency Count Method)
best case :



Insertion sort : best case
for ($i=1; i < n; i++$) — n
 $key = arr[i];$ — 1
 $j = i - 1;$ — 1
 while ($j >= 0 \ \&\& \ arr[j] > key$) — 1
 $arr[j+1] = arr[j];$ — 1
 $j--;$
 $arr[j+1] = key;$ — 1
 $T(n) = O(n)$

average case

Average case :

Pass-1 $\rightarrow n-1$ comparisons
Pass-2 $\rightarrow n-2$ comparisons
 $\therefore T(n) = \frac{n(n-1)}{2}$
ignore constants
 $= O(n^2)$

worst case :

Worst case : it occurs when
pivot is always smallest (&)
& largest
 $n + (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n+1)}{2}$
 $= O(n^2)$

SELECTION SORT ALGORITHM

Selection Sort :-
Input : Array A with n elements
Output : Sorted array A in ascending order

- 1) start
- 2) Read n and array A
- 3) Set $i=0$
- 4) set $min=i$
- 5) compare A[min] with the remaining elements
- 6) If $A[j] < A[min]$, set $min=j$
- 7) swap A[i] and A[min]
- 8) Increment i and repeat steps 4-7 until $i < n-1$.
- 9) Display sorted array A
- 10) stop.

PROGRAM:

```
#include <iostream> using
namespace std;

class SelectionSort { public:
int size, arr[100], i, j, temp;

    void arrayinput()
    {      cout << "Enter array
size: ";      cin >> size;

        cout << "Enter array elements: ";
for (i = 0; i < size; i++)
    {      cin
>> arr[i];
    }
    }

    void printarray()
    {      for (i = 0; i < size;
i++)
    {      cout <<
arr[i] << " ";
    }      cout
<< endl;
    }

    void selectionsortlogic()
    {      for (i = 0; i < size - 1;
i++)
    {      int
min = i;

        for (j = i + 1; j < size; j++)
        {      if (arr[j]
< arr[min])      {
min = j;
        }
    }
    }
```

```
        }
        if (min != i)
        {
            temp =
arr[i];
arr[i] = arr[min];    arr[min]
= temp;
        }
    }
}
};

int main()
{
    SelectionSort ss;

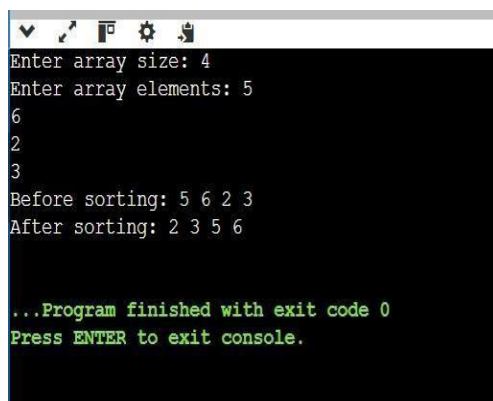
    ss.arrayinput();

    cout << "Before sorting: ";    ss.printarray();
ss.selectionsortlogic();

    cout << "After sorting: ";
ss.printarray();

    return 0;
}
```

OUTPUT :



```
Enter array size: 4
Enter array elements: 5
6
2
3
Before sorting: 5 6 2 3
After sorting: 2 3 5 6

...Program finished with exit code 0
Press ENTER to exit console.
```



time complexity : Step Count / Frequency Count Method)

best case :

```
for (i=0; i<n-1; i++) — n-1
{
    min=i; — 1
    for (j=i+1; j<n; j++) — n
    {
        if (arr[j]<arr[min]) — 1
        {
            min=j; — 1
        }
        y
    }
    temp = arr[i]; — 1
    arr[i] = arr[min]; — 1
    arr[min] = temp; — 1
    y
}
T(n) = n(n-1)
      = n2 - 2n + 1
      ~ O(n2)
```

average case :

```
for (i=0; i<n-1; i++) — n-1
{
    min=i; — 1
    for (j=i+1; j<n; j++) — n
    {
        if (arr[j]<arr[min]) — 1
        {
            min=j; — 1
        }
        y
    }
    temp = arr[i]; — 1
    arr[i] = arr[min]; — 1
    arr[min] = temp; — 1
    y
}
T(n) = n(n-1)
      = n2 - 2n + 1
      ~ O(n2)
```



worst case :

```
for (i=0; i<n-1; i++) — n-1
{
    min=i; — 1
    for (j=i+1; j<n; j++) — n
    {
        if (arr[j]<arr[min]) — 1
        {
            min=j; — 1
        }
        temp = arr[i]; — 1
        arr[i] = arr[min]; — 1
        arr[min] = temp; — 1
    }
    T(n) = n(n-1)
        = n2 - 2n + 1
        = O(n2)
}
```

MERGE SORT ALGORITHM :

4) Merge sort :-
Input : Array A with n elements
Output : Sorted array A in ascending order

- 1) start
- 2) Read n and array A
- 3) Divide the array into two halves
- 4) Recursively divide each half until single elements remain
- 5) compare elements of the two halves
- 6) merge them in sorted order
- 7) Repeat the merging until the complete array is sorted
- 8) Display sorted array A
- 9) stop.

PROGRAM:

```
#include <iostream> using
namespace std;

class mergesort
{ public:
    int size, arr[100], i;

    void arrayinput()
    {      cout << "Enter array
size: ";      cin >> size;
        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)      {      cin >> arr[i];
        }
    }

    void printarray()
    {      for (i = 0; i < size;
i++)
        {      cout <<
arr[i] << " ";
        }      cout
<< endl;
    }

    void merge(int low, int mid, int high)
    {      int temp[100];      int i =
low, j = mid + 1, k = low;

        while (i <= mid && j <= high)
        {      if (arr[i]
<= arr[j])
            {
temp[k] = arr[i];
i++;
            }      else
            {      temp[k] =
arr[j];      j++;
            }      k++;
    }
```



```
    }

    while (i <= mid)
    {
        temp[k]
= arr[i];
        i++;
        k++;
    }

    while (j <= high)
    {
        temp[k] = arr[j];    j++;    k++;
    }

    for (i = low; i <= high; i++)
    {
        arr[i] =
temp[i];
    }
}

void mergellogic(int low, int high)
{
    if (low <
high)
    {
        int mid = (low +
high) / 2;

        mergellogic(low, mid);
        mergellogic(mid + 1, high);

        merge(low, mid, high);
    }
}

};

int main() {
mergesort ms;

    ms.arrayinput();

    cout << "Before sorting: ";    ms.printarray();
}
```

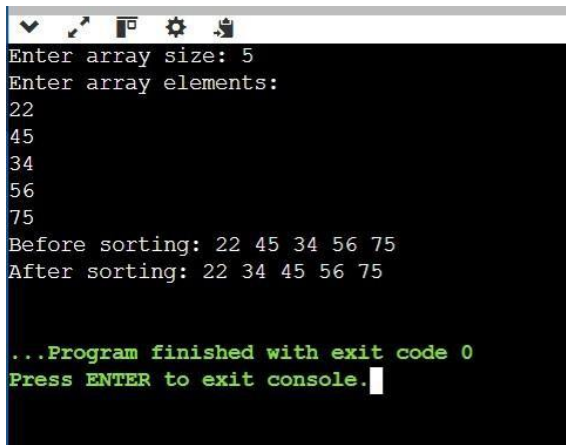
```

ms.mergelogsic(0, ms.size - 1);

cout << "After sorting: ";
ms.printarray();

return 0; }
OUTPUT :

```



```

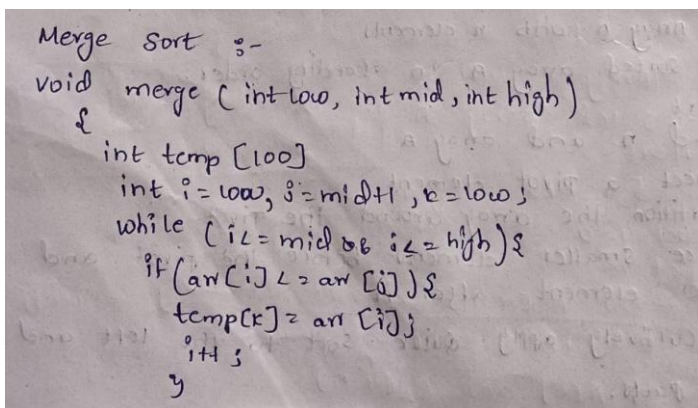
Enter array size: 5
Enter array elements:
22
45
34
56
75
Before sorting: 22 45 34 56 75
After sorting: 22 34 45 56 75

...Program finished with exit code 0
Press ENTER to exit console.

```

time complexity : Step Count / Frequency Count Method)

best case & average case & worst case :



```

Merge Sort :-
void merge (int low, int mid, int high)
{
    int temp[100];
    int i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
}

```

```

We{
  -I<O-.....C_:J:
  -u
  tr-tt-J
  ,,, C.,',&) f
  ..t;e....r (r.) : o.n l,J
  :1-1;
  t-tt-j
  ""1 c,,h:0h)t
  -t<"f [c] • " C.)
  +
  )
  ...1... Ca,...: :, h\oh*,..*)
  1
  "" c;) e l<~f:J;
  ,,,Jj,,,,,T< l n l u,,,;n{ f;ol,)
  : L10,,;L7h)
  "" <. (tow.\- hlo J/, , ) -r l n f,-j
  nle•11• l•-rc (u,u,,,;;&J _,, Tlh/•J
  =>(le l•Tic ( _,,,;<111, l,l,r")._); 1
  meu• t<ow' n>&, a)\
  "J

  ..J
  T(t)= ,t,/1.) t -r(")l..)-l-.,
  "T(nj:- tT( nJdu, -0
  7"J,..J-..n C"l + "l..-©
  Svb © :n 0
  "T(n) ""2-t tu("M) /2-)"
  J"lt> ) _ 1-f ( _,,,;t1.11 -0
  T(1) ur 1 "/,l)+- h ....;<9

  Sub (a) in (b)
  T(n) = 4(aT(n/16) + b/2) + 2n
  = 16T(n/16) + 2n + 2n
  = 16T(n/16) + 4n
  T(n) = 16T(n/16) + 4n

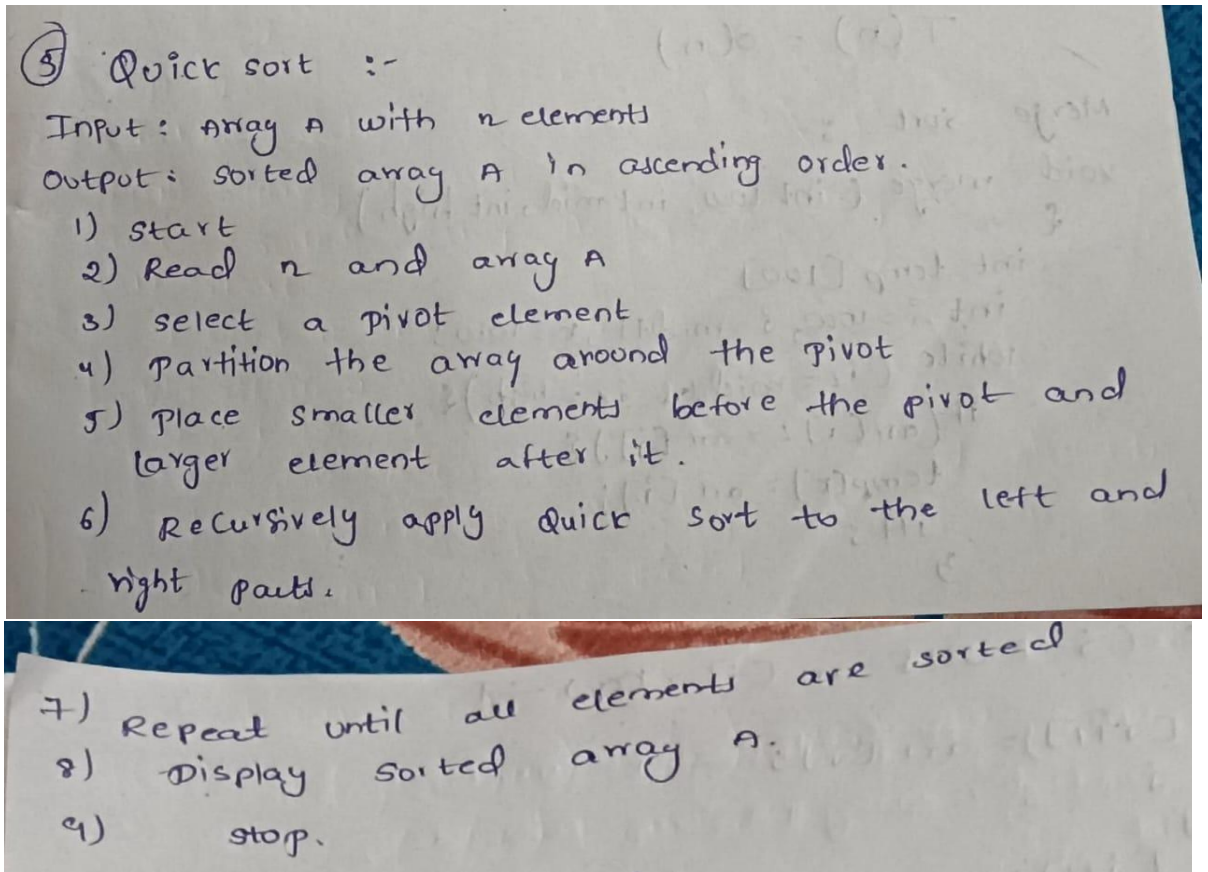
  = T( ) -1.t.th-
  T(.,) "- "2-"T ( "/-r::J+tC.,,
  "l, i

  -A-pp\j l<t) o,, b-.,,+, '11\Je.,
  "b n C- l'tJ ..
  ""d., -.. lC.
  "Tln) = .__1ou "(.) +Loa n(-,)
  "T(.,n ) "- r, l:oO '1-+- >l°C) )
  'IC,-..)» ( _1)+ nl°C),...,
  ,C, ....:) =- r' 100 ,.

```



QUICK SORT ALGORITHM:



PROGRAM :

```
#include <iostream> #include  
<utility> using  
namespace std;
```

```
class QuickSort {  
public: int size,  
arr[100];
```

```
    void arrayinput()  
    {        cout << "Enter array  
size: ";    cin >> size;  
  
        cout << "Enter array elements: ";  
        .....for (int i = 0; i < size; i++)
```



```
        {            cin
>> arr[i];
        }
    }

    void printarray()
    {            for (int i = 0; i < size;
i++)
        {            cout <<
arr[i] << " ";
        }            cout
<< endl;
    }

    int partition(int lb, int ub)
    {            int pivot =
arr[lb];
    int start = lb;
        int end = ub;

        while (start < end)
        {            while (start <= ub && arr[start]
<= pivot)                start++;

            while (arr[end] > pivot)
end--;
            if (start <
end)
                swap(arr[start], arr[end]);
        }

        swap(arr[lb], arr[end]);
    return end;
    }

    void quicksortlogic(int lb, int ub)
    {            if (lb < ub)            {
int loc = partition(lb, ub);
```

```
quicksortlogic(lb, loc - 1);  
quicksortlogic(loc + 1, ub);  
    }  
}
```

```
void sort() {  
quicksortlogic(0, size - 1);  
}  
};
```

```
int main()  
{
```

```
    QuickSort qs;
```

```
    qs.arrayinput();
```

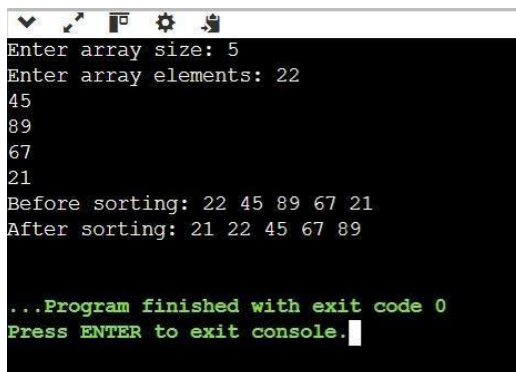
```
    cout << "Before sorting: ";  
    qs.printarray();
```

```
    qs.sort();
```

```
    cout << "After sorting: ";  
    qs.printarray();
```

```
    return 0;  
}
```

OUTPUT :



```
Enter array size: 5  
Enter array elements: 22  
45  
89  
67  
21  
Before sorting: 22 45 89 67 21  
After sorting: 21 22 45 67 89  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

TIME COMPLEXITY : Step Count / Frequency Count Method) best case :

Recurrence Relation

$$T(n) = 2T(n/2) + O(n)$$

expand

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

again

$$T(n) = 8T(n/8) + 3n$$

$$= 16T(n/16) + 4n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$n = 2^k$

add log on both sides

$$T(n) = 2 \log n (1) + \log n (n)$$

$$T(n) = n \log^2 + n \log n$$

$$T(n) = n(1) + n \log n$$

$$T(n) = n \log n$$

average case :

Recurrence Relation

$$T(n) = 2T(n/2) + O(n)$$

expand

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

again

$$T(n) = 8T(n/8) + 3n$$

$$= 16T(n/16) + 4n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$n = 2^k$

add log on both sides

$$T(n) = 2 \log n (1) + \log n (n)$$

$$T(n) = n \log^2 + n \log n$$

$$T(n) = n(1) + n \log n$$

$$T(n) = n \log n$$

worst case :

